

Questions

Q.1 AGENT RA Easy · Time limit 2 s

DBNO: 105376 Area: Greedy Algorithms LOD: 33

Problem Statement

Big *agent RA* is exploring *ancient ruins*. He found a *piece of paper* with a *word written on it*.

Fortunately, the people who used to live at this location several thousand years ago used *only two letters of the modern English alphabet: 'a' and 'b'*.

It's also known that *no ancient word contains two consecutive 'a's*.

RA has already recognized some of the *word's letters*, while others are still unknown.

RA wants to *find all valid words that could be written on this paper* based on the *ancient language dictionary*.

He needs your help to *find the word that is first in alphabetical order* and could be written on the paper.

Input Explanation

The first line contains a non-empty string **s** consisting of the characters '**a**', '**b**', and '**?**'. The character '**?**' represents an unrecognized letter.

It is **guaranteed** that **at least one valid ancient word** can be **written on the paper**.

Output Explanation

Output the word that is **first in alphabetical order** that could be **written on the paper**, found by **Agent RA**.

Constraints

Length of **s** is at **most 50**.

Sample Test Cases (3)

Example 1

Input

```
?ba??b
```

Output

```
ababab
```

Example 2

Input

```
a
```

Output

```
a
```

Example 3**Input**

????a

Output

ababba

Sample Test Case Explanation

Words **ababab**, **ababbb**, **bbabab** and **bbabbb** could be written on paper.

The first in **alphabetical order** is **ababab**.

Driver Code — Java

```
import java.util.Scanner;

public class Main {
    public static String ancientTranslation(String ancientWord) {
        //Write your code here
    }

    //Do not edit this part of code
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String ancientWord = scanner.next();
        System.out.println(ancientTranslation(ancientWord));
    }
}
```

Editorial

Solution is greedy. Traverse all character from left to right. Instead of every '?' try to put 'a' first, if two consecutive 'a's occurred put 'b'. You can always put 'b', it doesn't create two consecutive 'a's.

Putting 'a' first is correct, because all strings having 'a' at this position come earlier in alphabetical order, than all strings having the same prefix and having 'b' at the same position.

Code Editorial — Java

```
import java.util.Scanner;

public class Main {

    public static String ancientTranslation(String ancientWord) {
        StringBuilder s = new StringBuilder(ancientWord);
        int n = s.length();

        if (n == 0) {
            return "";
        } else if (n == 1) {
            if (s.charAt(0) == '?') {
                return "a";
            } else {
                return String.valueOf(s.charAt(0));
            }
        } else {
            for (int i = 0; i < n; i++) {
                if (s.charAt(i) == '?') {
                    if (i - 1 >= 0 && i + 1 < n) {
                        if (s.charAt(i - 1) == 'b' && (s.charAt(i + 1) == 'b' || s.charAt(i + 1) == '?')) {
                            s.replace(i, i + 1, "a");
                        } else {
                            s.replace(i, i + 1, "b");
                        }
                    }
                } else if (i - 1 < 0) {

```



```

        if (s.charAt(i + 1) == 'b' || s.charAt(i + 1) == '?') {
            s.replace(i, i + 1, "a");
        } else {
            s.replace(i, i + 1, "b");
        }
    } else if (i + 1 >= n) {
        if (s.charAt(i - 1) == 'b') {
            s.replace(i, i + 1, "a");
        } else {
            s.replace(i, i + 1, "b");
        }
    }
}

return s.toString();
}

}

// Do not edit this part of code
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    String ancientWord = scanner.next();
    System.out.println(ancientTranslation(ancientWord));
}
}

```

Hidden Test Cases (10)

Test Case 1 Level: Easy · Points: 5

Input

a?ab????ab???a???a???a???b???a???b????bba??b???ba?

Output

ababababbababbabababababababbababbababbbababababab

Test Case 2 Level: Easy · Points: 5

Input

a????????????????a?a?abb??ab????a?ba????ab????

Output

ababababababababababababbabababababbababbabababa

Test Case 3 Level: Easy · Points: 5

Input

???ba????aba????a?????a??a???bb????a???abb???

Output

abababababababababababababbababbbababbabababbaba

Test Case 4 Level: Medium · Points: 10

Input

??

Output

ab

Test Case 5 Level: Medium · Points: 10**Input**

aba

Output

aba

Test Case 6 Level: Medium · Points: 10**Input**

bbb

Output

bbb

Test Case 7 Level: Medium · Points: 10**Input**

?bb?

Output

abba

Test Case 8 Level: Hard · Points: 15**Input**

??b??

Output

abbab

Test Case 9 Level: Hard · Points: 15**Input**

?b?b?b?b?b

Output

ababababab

Test Case 10 Level: Hard · Points: 15**Input**

?b?b?b?b?b

Output

ababababab

Q.2 Sunehri and her Bag Hard · Time limit 1 s**DBNO: 105402 Area: Bit Manipulation LOD: 99****Problem Statement**

Sunehri is crossing a river over stepping stones, carrying a bag with N *integers*. Due to a sudden distraction, the bag falls and the integers scatter. Sunehri now wants to place these integers on N *stones*, such that each stone holds exactly one integer.

The **upset value** of the integer placed at index i (where $i \geq 2$) is calculated as:

$$\text{Upset}(i) = (A_i \wedge A_{i-1} \wedge A_{i-2}) \times P_i$$

where:

- A_i is the integer at the i -th stone after arrangement,
- P_i is the upset multiplier for the integer at index i ,
- $\wedge$ denotes the bitwise XOR operator.

For $i < 2$, the upset value is 0 .

Sunehri wants to minimize the **total upset value** over all stones. Help determine the optimal arrangement to minimize this total value.

Input Explanation

- The first line contains an integer T , the number of test cases.
- For each test case:
 - The first line contains an integer N .
 - The second line contains N integers representing array A .
 - The third line contains N integers representing array P .

Output Explanation

- For each test case, print the **minimum total upset value** in a new line.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 12$
- $0 \leq A_i, P_i \leq 200$

Sample Test Cases (2)

Example 1

Input

```
2
4
2 5 4 6
1 1 1 1
5
13 3 9 4 2
1 1 1 1 1
```

Output

```
1
14
```

Example 2

Input

```
1
2
100 200
1 2
```

Output

```
0
```

Sample Test Case Explanation

Explanation: Test Case 1

Optimal arrangement: 5, 6, 2, 4

- Upset for position 0 and 1: 0
- Upset at index 2: $(5^6 \times 2) \times 1 = 1$
- Upset at index 3: $(6^2 \times 4) \times 1 = 0$
- Total = $0 + 0 + 1 + 0 = 1$

Optimal arrangement: 3, 2, 9, 13, 4

- Upset for position 0 and 1: 0
- Upset at index 2: $(3^2 \times 9) \times 1 = 8$
- Upset at index 3: $(2^9 \times 13) \times 1 = 6$
- Upset at index 4: $(9^{13} \times 4) \times 1 = 0$
- Total = $0 + 0 + 8 + 6 + 0 = 14$

Driver Code — Java

```
import java.util.*;

public class Main {
    static List<Integer> calculateMinimum(int T, List<Integer> N, List<List<Integer>> A, List<List<Integer>> P) {
        //Write your code here
    }

    //Do not edit this part of code
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int T = scanner.nextInt();
        List<Integer> N = new ArrayList<>();
        List<List<Integer>> A = new ArrayList<>();
        List<List<Integer>> P = new ArrayList<>();
        for (int i = 0; i < T; ++i) {
            N.add(scanner.nextInt());
            List<Integer> currentA = new ArrayList<>();
            List<Integer> currentP = new ArrayList<>();
            for (int j = 0; j < N.get(i); ++j) {
                currentA.add(scanner.nextInt());
            }
            for (int j = 0; j < N.get(i); ++j) {
                currentP.add(scanner.nextInt());
            }
            A.add(currentA);
            P.add(currentP);
        }

        List<Integer> results = calculateMinimum(T, N, A, P);
        for (int res : results) {
            System.out.println(res);
        }
    }
}
```

Editorial

Explanation: Test Case 1

Optimal arrangement: 5, 6, 2, 4

Upset for position 0 and 1: 0

Upset at index 2: $(5^6 \times 2) \times 1 = 1$

Upset at index 3: $(6^2 \times 4) \times 1 = 0$

Total = $0 + 0 + 1 + 0 = 1$

Optimal arrangement: 3, 2, 9, 13, 4

Upset for position 0 and 1: 0

Upset at index 2: $(3^2 \times 9) \times 1 = 8$

Upset at index 3: $(2^9 \times 13) \times 1 = 6$

Upset at index 4: $(9^{13} \times 4) \times 1 = 0$

Total = $0 + 0 + 8 + 6 + 0 = 14$

Code Editorial — Java

```
import java.util.*;

public class Main {

    static int dp[][][][];
    static int[] a;
    static int[] p;
    static int n;

    // DP function for calculating the minimum value
    static int Dp(int l, int i, int j, int m) {
        if (l == n) return 0;
        if (dp[l][i][j][m] != -1) return dp[l][i][j][m];
        int min = Integer.MAX_VALUE;
        for (int k = 0; k < n; k++) {
            if ((m & (1 << k)) == 0)
                min = Math.min(min, Dp(l + 1, j, k, (m | (1 << k))) + (a[i] ^ a[j] ^ a[k]) * p[l]);
        }
        return dp[l][i][j][m] = min;
    }

    // Function to calculate the minimum for each test case
    static List<Integer> calculateMinimum(int T, List<Integer> N, List<List<Integer>> A, List<List<Integer>> P) {
        List<Integer> results = new ArrayList<>();

        for (int g = 0; g < T; g++) {
            n = N.get(g);
            a = new int[n];
            p = new int[n];

            // Initialize dp table
            dp = new int[n][n][n][1 << 13];
            for (int i = 0; i < p.length; i++) {
                for (int j = 0; j < p.length; j++) {
                    for (int k = 0; k < p.length; k++) {
                        Arrays.fill(dp[i][j][k], -1);
                    }
                }
            }

            // Input the values for A and P
            for (int i = 0; i < n; i++) {
                a[i] = A.get(g).get(i);
            }
            for (int i = 0; i < n; i++) {
                p[i] = P.get(g).get(i);
            }

            // Start calculating the minimum value
            int ans = Integer.MAX_VALUE;
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (i != j) {
                        ans = Math.min(ans, Dp(2, i, j, (1 << i) | (1 << j)));
                    }
                }
            }
            results.add(ans);
        }
    }
}
```

```
    }

    return results;
}

// Main function for reading input and printing results
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int T = scanner.nextInt();
    List<Integer> N = new ArrayList<>();
    List<List<Integer>> A = new ArrayList<>();
    List<List<Integer>> P = new ArrayList<>();

    for (int i = 0; i < T; ++i) {
        N.add(scanner.nextInt());
        List<Integer> currentA = new ArrayList<>();
        List<Integer> currentP = new ArrayList<>();
        for (int j = 0; j < N.get(i); ++j) {
            currentA.add(scanner.nextInt());
        }
        for (int j = 0; j < N.get(i); ++j) {
            currentP.add(scanner.nextInt());
        }
        A.add(currentA);
        P.add(currentP);
    }

    // Get the results from the calculateMinimum function
    List<Integer> results = calculateMinimum(T, N, A, P);

    // Output results for each test case
    for (int res : results) {
        System.out.println(res);
    }

    scanner.close();
}
}
```

Hidden Test Cases (5)**Test Case 1** Level: Easy · Points: 10**Input**

```
1
8
198 27 91 82 56 167 186 6
23 41 182 10 160 79 161 129
```

Output

```
35861
```

Test Case 2 Level: Medium · Points: 20**Input**

```
10
5
10 49 9 34 32
1 1 1 1 1
5
5 31 20 19 29
1 1 1 1 1
5
35 8 36 39 14
1 1 1 1 1
5
```

```
17 15 27 0 23
1 1 1 1 1
5
3 9 36 23 2
1 1 1 1 1
5
2 28 43 38 17
1 1 1 1 1
5
27 27 38 3 2
1 1 1 1 1
5
45 48 48 30 20
1 1 1 1 1
5
49 48 15 41 35
1 1 1 1 1
5
24 33 32 36 10
1 1 1 1 1
```

Output

```
43
18
31
18
73
70
66
15
41
50
```

Test Case 3 Level: Hard · Points: 30

Input

```
10
8
19 100 147 184 155 97 67 129
105 7 53 172 134 85 143 100
8
198 27 91 82 56 167 186 6
23 41 182 10 160 79 161 129
8
180 57 62 84 154 130 163 58
87 15 180 20 50 73 71 47
8
50 162 80 106 79 65 113 102
56 44 112 166 123 72 95 102
8
80 107 136 33 187 98 42 24
63 21 44 114 44 65 111 95
8
178 141 0 56 5 63 108 62
107 19 27 30 41 72 82 121
8
130 18 105 116 66 97 140 130
68 135 194 113 150 54 7 77
8
196 158 83 151 21 191 163 128
160 141 108 1 12 191 72 92
8
159 127 159 175 23 48 54 92
133 47 155 83 52 162 110 47
8
```

```
69 194 198 90 184 161 169 144
51 26 95 13 167 117 106 125
```

Output

```
36550
35861
12581
30629
18337
8899
30304
5397
20343
17182
```

Test Case 4 Level: Hard · Points: 30**Input**

```
10
5
10 49 9 34 32
1 1 1 1 1
5
5 31 20 19 29
1 1 1 1 1
5
35 8 36 39 14
1 1 1 1 1
5
17 15 27 0 23
1 1 1 1 1
5
3 9 36 23 2
1 1 1 1 1
5
2 28 43 38 17
1 1 1 1 1
5
27 27 38 3 2
1 1 1 1 1
5
45 48 48 30 20
1 1 1 1 1
5
49 48 15 41 35
1 1 1 1 1
5
24 33 32 36 10
1 1 1 1 1
```

Output

```
43
18
31
18
73
70
66
15
41
50
```

Test Case 5 Level: Easy · Points: 10**Input**

```
2
4
2 5 4 6
1 1 1 1
5
13 3 9 4 2
1 1 1 1 1
```

Output

```
1
14
```